

axini

Model Based Testing with the **Axini Modeling Platform**

08/10/2026 – UvA SSVT

Axini lecture

L1.02	13.00–13.45	Lecture: introduction to Axini, coffee machine
	13.45–14.00	<i>break</i>
	14.00–14.45	Lecture: more on AML, SmartDoor introduction
L1.17	15.00–16.00	Laptop college: hands-on SmartDoor teams 1–8
	16.00–17.00	Laptop college: hands-on SmartDoor teams 9–16

Machiel van der Bijl



Susan van den Broek



- **scale-up:**

- ~20 years
- 25+ employees
 - developers – Ruby on Rails
 - test architects / consultants

But simply called
'**Axini**' by our users.

- product: **Axini Modeling Platform**

- based on conformance testing theory of Tretmans et.al.

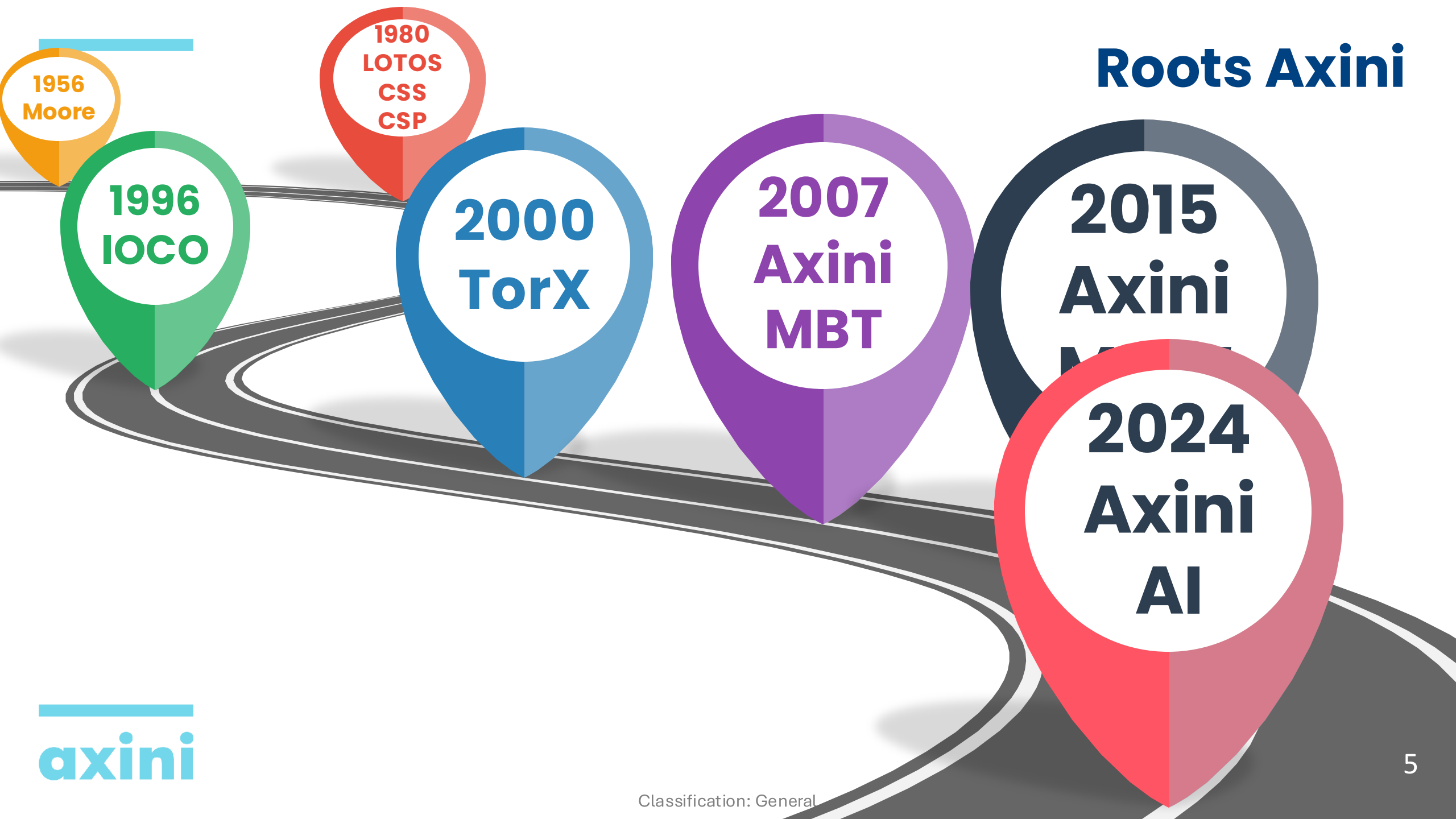
- AMP is used to test

- **technical** applications (e.g., client: ProRail, Thermo Fisher)
- **financial** applications (e.g., client: Achmea)

Our **goal** is to help clients deliver:

- software products that **work** when it matters,
- **on time**, without concessions on quality, using our **unique testing platform**.

Roots Axini



Model-Based Testing

and how do to it in AMP

Many different testing approaches

*orthogonal and
complementary*

- unit testing
- test/behavior driven development (BDDs)
- input-output testing
- mutation testing
- classification testing
- interface testing
- integration testing
- non-functional testing
- regression testing
- user acceptance testing
- maintenance testing
- **model based testing**

*Except for MBT, for all these approaches the test cases must be **written manually**. With **MBT**, test cases are automatically **generated**.*

Best choice for testing:

- reactive, control-oriented systems
- several interfaces
- multi-threaded, parallel processes

Black-box testing: we do not have the source of the System under Test (SUT)!

Model Based Testing

is an **automated** technical process,

... performed by **automatically** executing/experimenting

... with a **software** system, in a **controlled environment**,

... following prescribed behavior of a **formal model**,

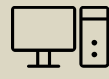
... with the intent of **measuring** one or more **characteristics** or the **quality** of the product

... by demonstrating the **deviation** of the actual status of the product from the required status **specification**.

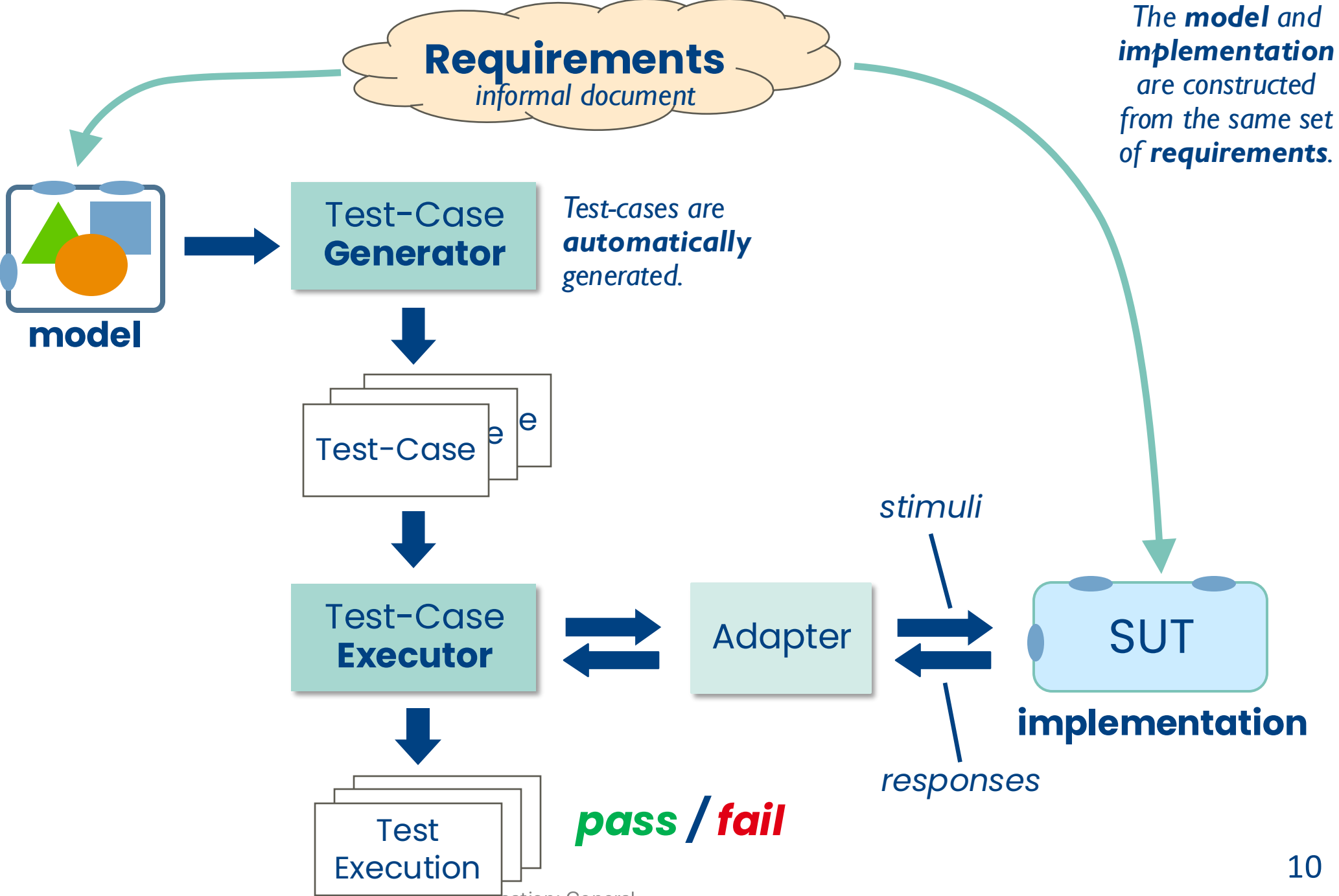
The '*Model*' is an **abstract**, **high-level** and **formal** description of the SUT.

Test tool comparison

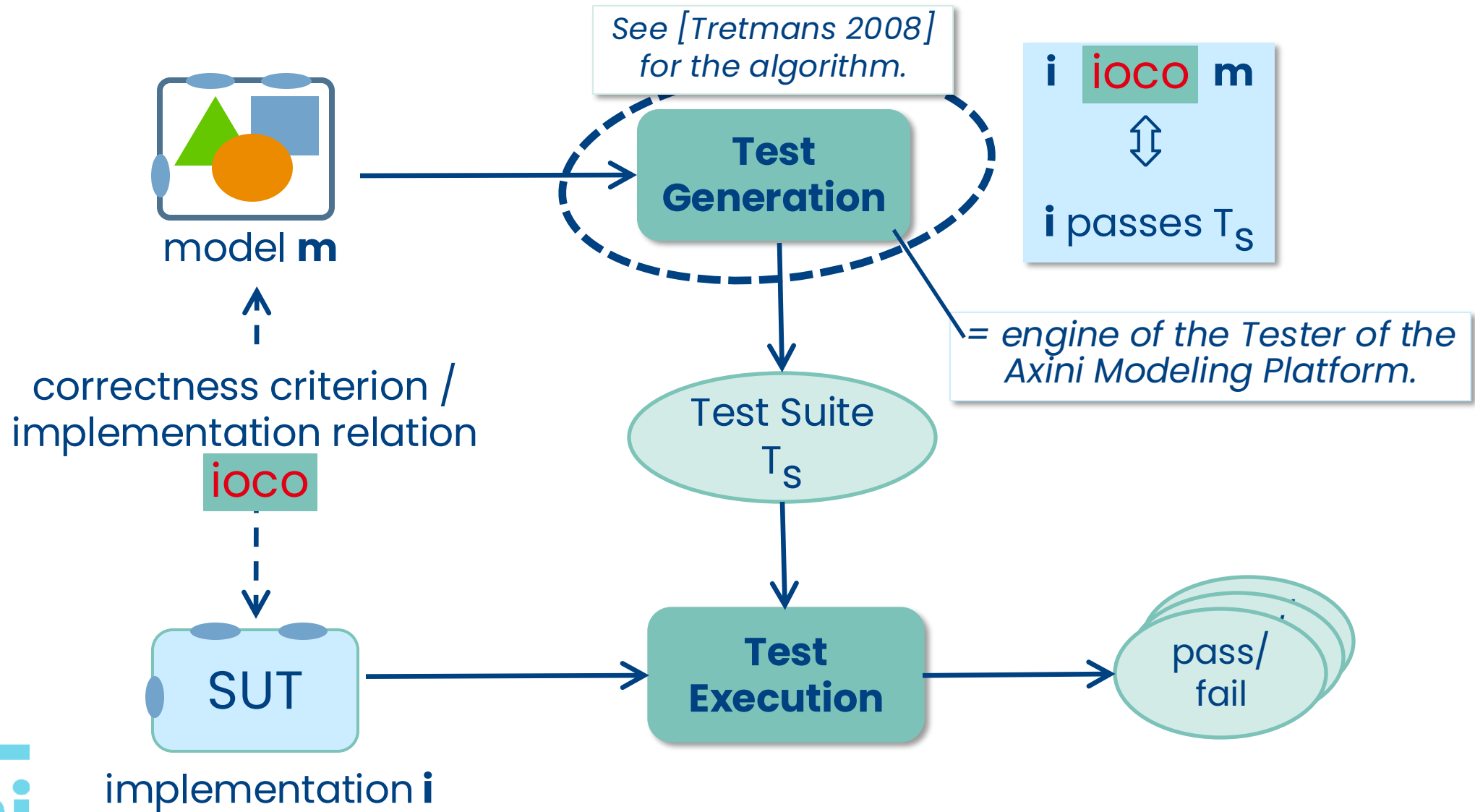
 Manual step
 Automated

	Process	Manual	BDD	Axini
Design	Make specification			 
	Make model			
Test	Make test			
	Predict outcome			
	Script test			
	Execute test			
	Evaluate outcome			

Model Based Testing



Formal Conformance Testing

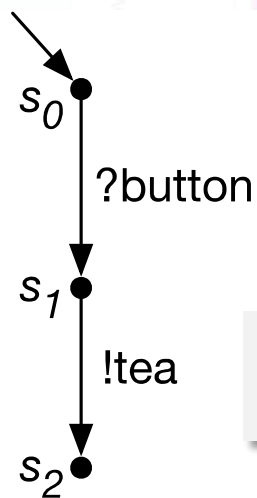


AMP: Axini Modeling Platform

LTS is the model... but not very convenient for writing large models.

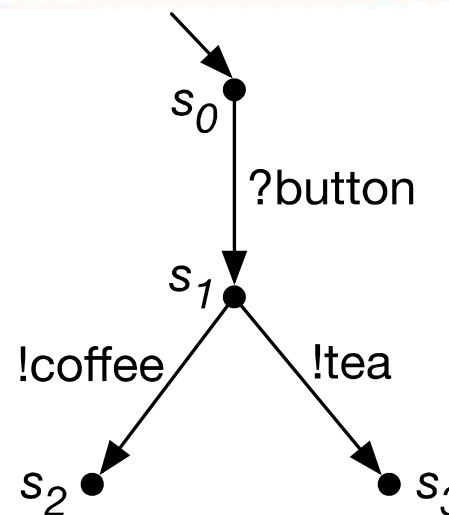
- **AMP**: tool to edit, explore, visualize, test AML models
 - **Test-cases** are automatically generated.
- Models are written in the Axini Modeling Language (**AML**)
 - **behavior**: mapped upon **labeled transition systems (LTSs)**
 - **data**: modern, powerful, static typed 'programming' language

Examples of LTSs



AML

```
receive 'button'  
send 'tea'
```



AML

```
receive 'button'  
choice {  
  o { send 'coffee' }  
  o { send 'tea' }  
}
```

? = input (*stimulus*)
! = output (*response*)

Axini Modeling Language (AML)

- process + data language
 - inspired by **Promela** (SPIN) and **LOTOS**
- model consists of **parallel processes**
- communication over **hand-shake channels**
 - *external*: communication with SUT
 - *internal*: communication between processes

Largest AML
model: >10k loc.

Semantics

A process
in AML is
mapped
upon a
(*symbolic*)
labelled
transition
system.

behavioral part:

- **stimuli** (inputs)
- **responses** (outputs)
- (non-deterministic) **choice**
- **repeat**
- states / **goto**

first things first

data part:

- Ruby-like, **strictly typed**
- messages can be **received** and **send**
 - **label** (name)
 - **parameters** (attributes)
- process can have **variables**

AML is implemented as a **Ruby DSL**: Ruby can be used as preprocessor.

Hello, Tea Machine

```
# Hello Tea: our first AML model!
```

```
external 'machine' ——— name of the external channel
```

```
process('hello-tea') {
```

```
  # declarations of labels, variables
```

```
  timeout 10.0 ——— timeout for all responses
```

```
  stimulus 'button', on: 'machine'
```

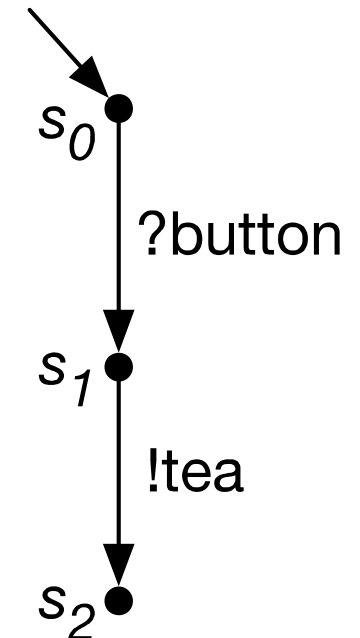
```
  response 'tea', on: 'machine'
```

```
  # behavior of the process
```

```
    receive 'button'
```

```
    send 'tea'
```

```
}
```

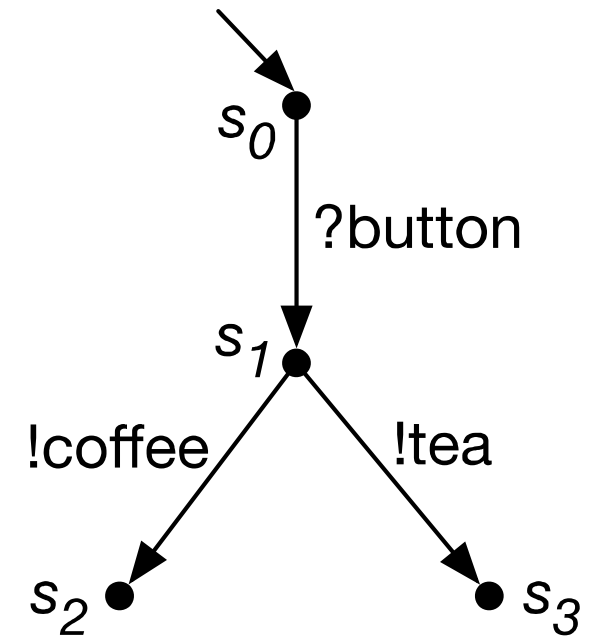


choice

```
external 'extern'
process('tea-or-coffee') {
  # declarations of labels, variables
  timeout 10.0
  channel('extern') {
    stimulus 'button'
    responses 'tea', 'coffee'
  }

  # behavior of the process
  receive 'button'
  choice {
    o { send 'tea' }
    o { send 'coffee' }
  }
}
```

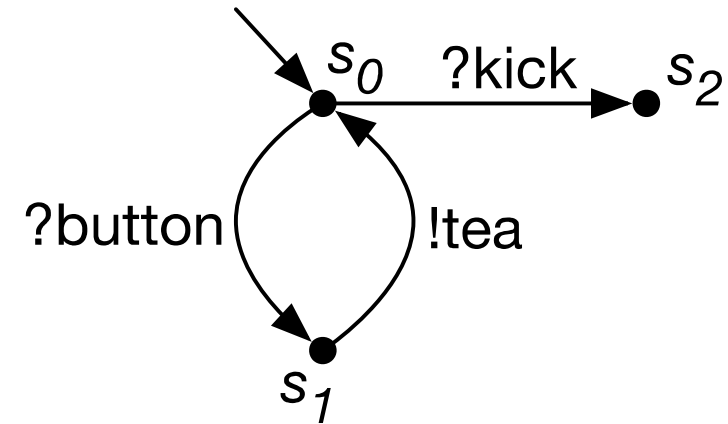
declarations keep
(more-or-less) the same



states and goto

```
external 'extern'
process('tea-or-kick-goto') {
  timeout 10.0
  channel('extern') {
    stimuli 'button', 'kick'
    responses 'tea'
  }

  state 's0'
  choice {
    o { receive 'kick' }
    o { receive 'button'; send 'tea'; goto 's0' }
  }
}
```

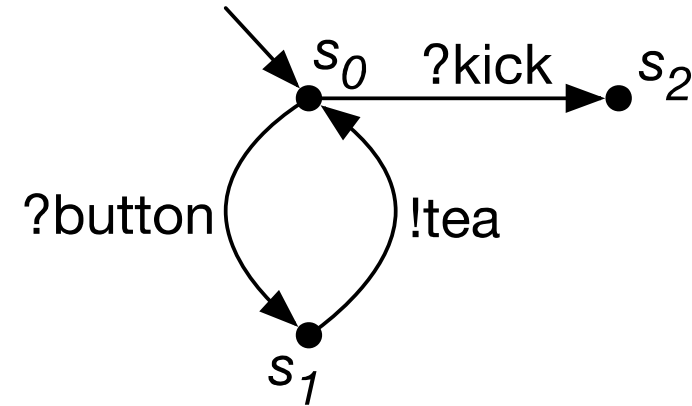


*In programming languages the **goto** statement might be not-done.
In modelling languages (for reactive systems) it is a powerful
mechanism to model **state-transition systems**, such as **protocols**.*

repeat

```
external 'extern'
process('tea-or-kick-repeat') {
  timeout 10.0
  channel('extern') {
    stimuli 'button', 'kick'
    responses 'tea'
  }

  repeat {
    o { receive 'kick' ; stop_repetition }
    o { receive 'button'; send 'tea' }
  }
}
```



break out of the loop

axini

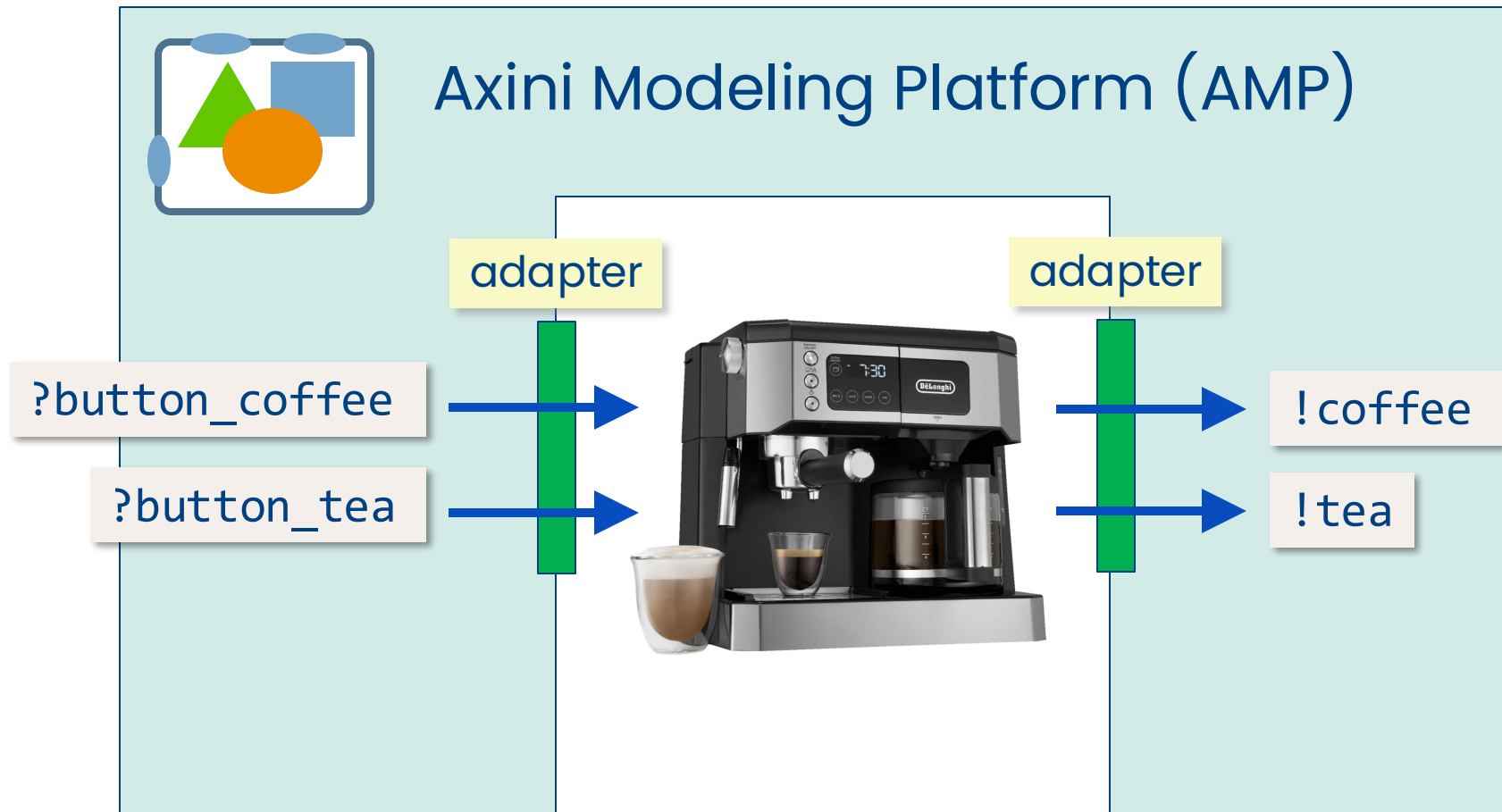
Coffee Machine

System Under Test



Challenge: **model** and **test** that the beverage machine delivers **coffee**, **tea**, and ... **lemonade**.

AMP: 'horseshoe' around SUT



AMP + model + adapters
'simulate' the **environment** of the SUT.

Let's do it together

- (coffee machine demo on course02)

Coffee Machine

```
state 'start'
  choice {
    o { receive 'button_coffee' ; goto 'coffee' }
    o { receive 'button_tea' ; goto 'tea' }
    o { receive 'button_lemonade' ; goto 'lemonade' }
  }

state 'coffee'
  send 'coffee'
  goto 'start'

state 'tea'
  send 'tea'
  goto 'start'

state 'lemonade'
  choice {
    o { send 'lemonade' }
    o { send 'coffee' }
    o { send 'tea' }
  }
  goto 'start'
```

After *?button_lemonade*,
we observe either
!lemonade, *!coffee*, or *!tea*.

Using *repeat* instead of states/goto.

```
repeat {
  o { receive 'button_coffee'; send 'coffee' }
  o { receive 'button_tea'; send 'tea' }
  o { receive 'button_lemonade'
    choice {
      o { send 'coffee' }
      o { send 'tea' }
      o { send 'lemonade' }
    }
  }
}
```

```
state 'start'
  choice {
    o { receive 'button_coffee' }
    o { receive 'button_tea' }
    o { receive 'button_lemonade' }
  }

  choice {
    o { send 'coffee' }
    o { send 'tea' }
    o { send 'lemonade' }
  }
  goto 'start'
```

Too **loose**: allowing
too much behavior.

Coffee Machine (exact?)

Observation: after
?button_lemonade, the **last** given
beverage is observed **again**.

```
state 'start'
  choice {
    o { receive 'button_coffee' ; send 'coffee' ; goto 'coffee_last' }
    o { receive 'button_tea' ; send 'tea' ; goto 'tea_last' }
    o { receive 'button_lemonade' ; send 'lemonade' ; goto 'start' }
  }

state 'coffee_last'
  choice {
    o { receive 'button_coffee' ; send 'coffee' ; goto 'coffee_last' }
    o { receive 'button_tea' ; send 'tea' ; goto 'tea_last' }
    o { receive 'button_lemonade' ; send 'coffee' ; goto 'coffee_last' }
  }

state 'tea_last'
  choice {
    o { receive 'button_coffee' ; send 'coffee' ; goto 'coffee_last' }
    o { receive 'button_tea' ; send 'tea' ; goto 'tea_last' }
    o { receive 'button_lemonade' ; send 'tea' ; goto 'tea_last' }
  }
```

Only at the start,
!lemonade is
observed.

Coffee Machine (exact?)

```
var 'last', :string, ''
```

```
repeat {
```

```
  o { receive 'button_coffee'; send 'coffee', update: "last = 'coffee'" }
```

```
  o { receive 'button_tea'; send 'tea', update: "last = 'tea'" }
```

```
  o {
```

```
    receive 'button_lemonade'
```

```
    choice {
```

```
      o { send 'lemonade', constraint: "last == ''" }
```

```
      o { send 'tea', constraint: "last == 'tea'" }
```

```
      o { send 'coffee', constraint: "last == 'coffee'" }
```

```
    }
```

```
  }
```

```
}
```

Using a **state variable**, which remembers the last beverage.

We also have to use the **update** and **constraint** options of a label here.

axini

Some more AML

non-terminating behavior $\approx$ state

```
behavior('start', :non_terminating) {  
  choice {  
    o { receive 'button_coffee' ; send 'coffee' ; behave_as 'coffee_last' }  
    o { receive 'button_tea' ; send 'tea' ; behave_as 'tea_last' }  
    o { receive 'button_lemonade' ; send 'lemonade' ; behave_as 'start' }  
  }  
}
```

behave_as $\approx$ goto

```
behavior('coffee_last', :non_terminating) {  
  choice {  
    o { receive 'button_coffee' ; send 'coffee' ; behave_as 'coffee_last' }  
    o { receive 'button_tea' ; send 'tea' ; behave_as 'tea_last' }  
    o { receive 'button_lemonade' ; send 'coffee' ; behave_as 'coffee_last' }  
  }  
}
```

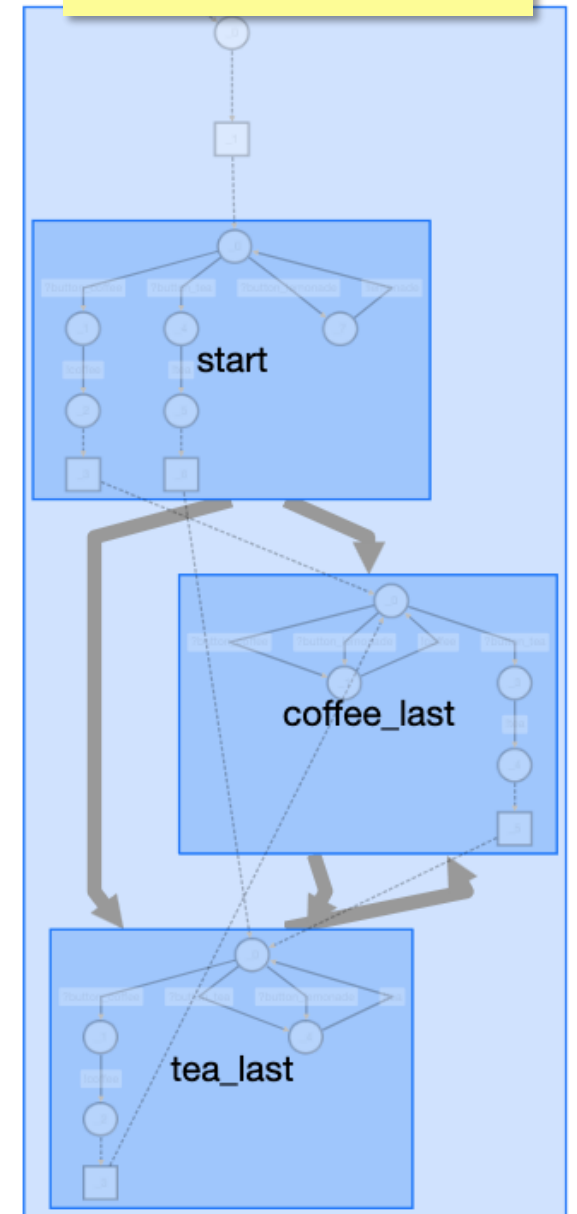
```
behavior('tea_last', :non_terminating) {  
  choice {  
    o { receive 'button_coffee' ; send 'coffee' ; behave_as 'coffee_last' }  
    o { receive 'button_tea' ; send 'tea' ; behave_as 'tea_last' }  
    o { receive 'button_lemonade' ; send 'tea' ; behave_as 'tea_last' }  
  }  
}
```

```
behave_as 'start'
```

Visualization of non-terminating behaviors is nicer than states.

*By some of our clients, **non-terminating behaviors** are preferred over states/goto.*

Behaviors get their own 'blocks'.



Axini Modeling Language (AML)

- process + data language
 - inspired by **Promela** (SPIN) and **LOTOS**
- model consist of **parallel processes**
- communication over **hand-shake channels**
 - *external*: communication with SUT
 - *internal*: communication between processes

Largest AML
model: >10k loc.

Semantics

A process
in AML is
mapped
upon a
(*symbolic*)
labelled
transition
system.

behavioral part:

- **stimuli** (inputs)
- **responses** (outputs)
- (non-deterministic) **choice**
- **repeat**
- states / **goto**

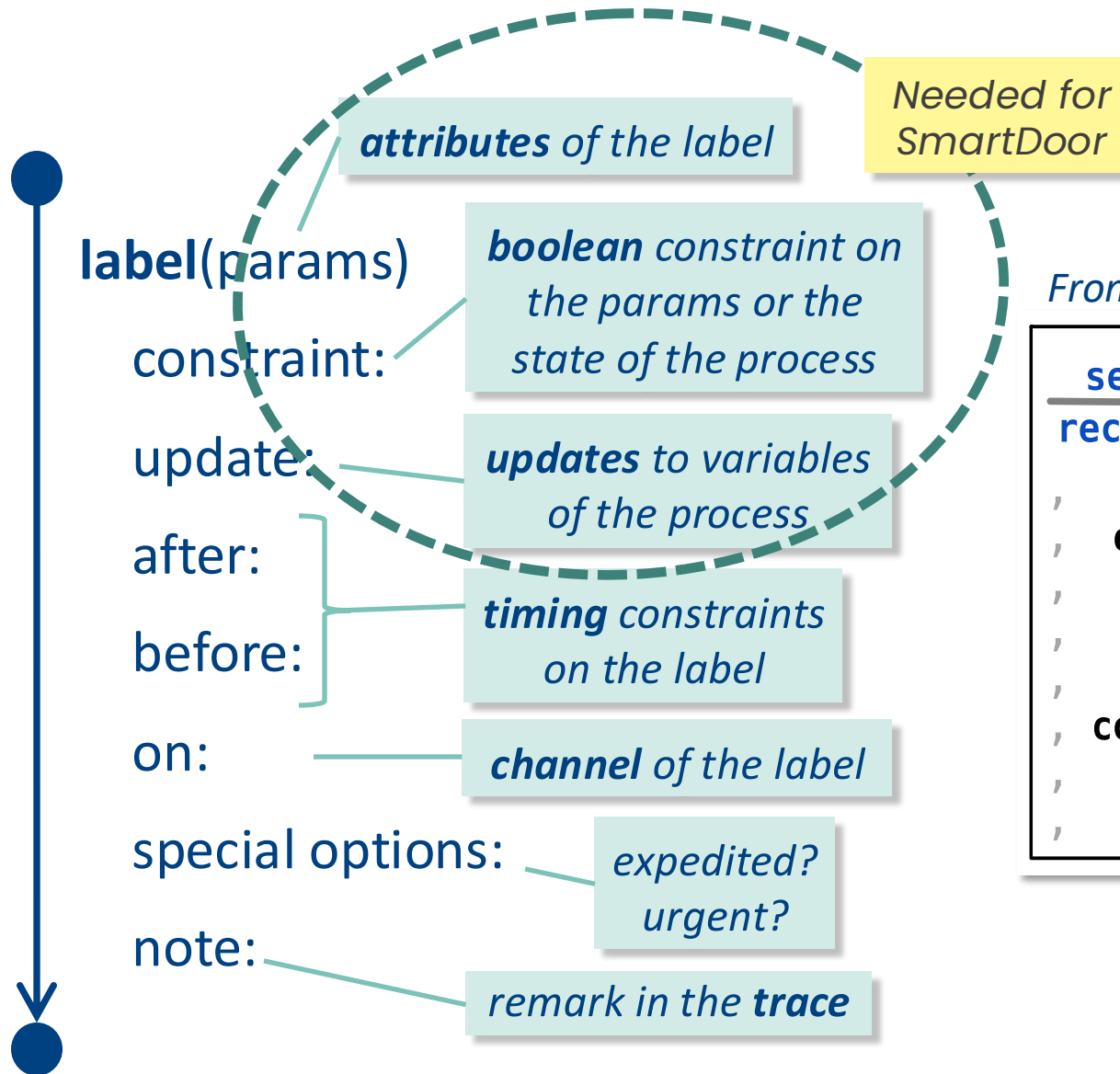
data part:

- Ruby-like, **strictly typed**
- messages can be **received** and **send**
 - **label** (name)
 - **parameters** (attributes)
- process can have **variables**

AML is implemented as a **Ruby DSL**: Ruby can be used as preprocessor.

Label in AML

A label is **more** than an **observable action**.



From AML Quick Reference Card:

```
send
receive label_name
, on: channel_name
, expedited: <boolean>
, urgent: <boolean>
, after: <time_expr> | <decimal_expr>
, before: <time_expr> | <decimal_expr>
, constraint: <bool_expr>
, update: <assignment_expr>
, note: <string> | [ <string>* ]
```

all label options are optional

For a label to be *enabled*, its *constraint* must be *true*.

Label parameters

```
external 'extern'
process('coin-tea-parameters') {
  timeout 10.0
  channel('extern') {
    stimulus 'coin', {'value' => :integer}
    response 'tea', {'volume' => :integer}
  }

  repeat {
    o {
      receive 'coin', constraint: 'value == 50'
      send 'tea', constraint: 'volume == 200'
    }
    o {
      receive 'coin', constraint: 'value == 100'
      send 'tea', constraint: 'volume == 500'
    }
  }
}
```

parameters have a **name** and **type**

creates stimulus with specific parameter

checks parameters of the response

AMP will **generate** values for the label **parameters** such that the **constraint** is **true**.

State variables

```
external 'extern'
process('coin-tea-total') {
  timeout 10.0
  channel('extern') {
    stimulus 'coin',      {'value' => :integer}
    stimulus 'stop'
    response 'tea',       {'volume' => :integer}
    response 'display',   {'number' => :integer}
  }
  var total', :integer, 0
  repeat {
    o {
      receive 'coin', constraint: 'value == 50',
        update: 'total = total + value'
      send 'tea',      constraint: 'volume == 200'
    }
    o {
      receive 'coin', constraint: 'value == 100',
        update: 'total = total + value'
      send 'tea',      constraint: 'volume == 500'
    }
    o { receive 'stop'; stop_repetition }
  }
  send 'display', constraint: 'number == total'
}
```

state variable

update of state variables

variables & types

All **identifiers** in AML are **strings** (between **single** or **double** quotes).

- **label parameters** *curly brackets are optional*

```
stimulus '<name>', { '<name>' => <type>, ... }  
response '<name>', { '<name>' => <type>, ... }
```

- **state variables** – local to a process

```
var '<name>', <type>, <initial_value>
```

The initial value is **optional**. If not provided, the value will be **:void** (equivalent to null/nil).

- **types**

- simple types `:integer :decimal :string :boolean :time :date`

- structured types

- array `[ <type> ]`

- struct `{ '<name>' => <type>, ... }`

- hash `{ <type> => <type> }`

AML does **not** support global variables, **by design**.

variable usage

- **constraint** of a transition/action:

response

```
send 'value', constraint: 'x > 3 && y < 4'
```

- **x, y** can be label parameters of 'value', or state variables; send is only enabled when the constraint evaluates to **true**.

stimulus

```
receive 'dice', constraint: 'x > 3'
```

- If **x** is a label parameter of 'dice', AMP will **generate** a value for **x** such that the expression 'x > 3' is **true**.
- If **x** is a state variable, the expression 'x > 3' must evaluate to **true** for the receive to be enabled.

- **update** of a transition/action:

stimulus/receive

```
send 'prize', update: 'prize_won = true'
```

- variable 'prize_won' must be a boolean state variable.

stimuli: label parameters

AMP requires that **all label parameters** of a **stimulus** are **constrained**.

```
external 'extern'
process('stimulus-example') {
  timeout 10.0
  channel('extern') {
    stimulus 'input', { 'x' => :integer }
    response 'output', { 'y' => :integer, 'z' => :string }
  }
}
```

nil value

```
receive 'input', constraint: 'x != :void'
```

any value (but AMP will probably choose 0, ... the first time)

```
receive 'input', constraint: 'x == 42'
```

fixed value

```
receive 'input', constraint: 'x >= 0 && x <= 10'
```

interval

```
receive 'input', constraint: 'x in [0, 2, 4, 6, 8]'
```

from a list

```
send 'output', constraint: 'y == 42'
}
```

response: check on y, but not on z

Example

```
external 'external'
process('parameters-example') {
  timeout 10.0
  channel('external') {
    stimulus 'input', { 'y' => :integer }
    response 'output', { 'z' => :integer }
  }
}
```

```
var 'x', :integer, 2
var 'y_rcv', :integer
var 'z_snd', :integer
```

```
receive 'input',
  constraint: 'y > 0 && y < 10 && x > 0',
  update: 'y_rcv = y; x += 10'
```

```
send 'output',
  constraint: 'z > y_rcv && x > 10',
  update: 'z_snd = z'
```

```
}
```

variables to **store** the values of label parameters

constraint on label parameter

constraint on state variable

AMP will thus **generate** a value for **y** between 0 and 10.

saving the value of the label parameter into a state variable

AML – Advanced Features

- **structured data types** (lists, structs, hashes)
- **behavior definitions** to encapsulate behavior
- **internal communication** between processes
- **timed testing** features (delay and force labels)
- **function definitions** for complex computations
- **urgent transitions** to enforce internal communication
- use of **Ruby** as preprocessor to structure the model

*... see the "Modeling Tutorial"
in the documentation of AMP.*

+ Reference material:

- AML Reference Manual
- AML Quick Reference

Your MSc thesis at Axini

Formal methods & verification

Model checking of symbolic transition systems – Automatically verify model properties with linear time logic and formal methods

Requirements coverage – Formalize natural language to guarantee that requirements are covered by models and test cases

Debugger – Research and implement advanced debugging functionality

Testing & coverage

Data coverage – Define, monitor and optimize data coverage in model-based testing

Time-bound testing – Extend AMP and symbolic transition systems with time as a factor

Model-based testing of GUI applications – Test web, desktop and mobile applications with MBT theory

Modeling & visualization

Visualization of models – Make large, complex models and concepts like parallelism and non-determinism visually insightful

Visual modeling – Combine textual and visual modeling via a shared representation

Design a modeling language for data and interaction – Develop a new language for reactive and data-oriented systems

AI & automation

Model learning – Generate formal models based on observed behavior or log files

Automated model generation – Convert existing models or code into formal AML models

Code generation from formal models – Generate application code directly from formal models

Your MSc thesis at Axini

- Check out <https://axini.com/studenten>
- Interested? Send an email to students@axini.com
- Or visit us during the Thesis Fair!

🏠 ISSTA/ECOOP 2024 (series) / DEBT 2024 (series) /

Second Workshop on Future Debugging Techniques

About Program Accepted Papers Call for Papers

Accepted Papers

★ Title

★ Abstract Debugging with GobPie
Karoline Holter, Juhan Oskar Hennoste, Simmo Saan, Patrick Lam, Vesal Vojdani
DOI Media Attached

★ A Reversible Debugger for MPI Applications
Mihkel Tiks, Ott-Kaarel Martens, Eero Vainikko, Stefan Kuhn
DOI

★ Bug localisation in Model-Based Testing Using Model-Centric Shrinking
Susan van den Broek, Ana Oprescu, Theo C Ruys
DOI

Important Dates

Thu 19 Sep 2024
DEBT Workshop

Wed 31 Jul 2024
Camera ready

Mon 22 Jul 2024
Author Notification

Fri 5 Jul 2024
Demo Submission Deadline

Fri 28 Jun 2024
Paper Submission Deadline

Bug Localisation in Model-Based Testing using Model-Centric Shrinking

Susan van den Broek
susan.vandenbroek@student.uva.nl
University of Amsterdam
Amsterdam, The Netherlands

Ana Oprescu
a.m.oprescu@uva.nl
University of Amsterdam
Amsterdam, The Netherlands

Theo C. Ruys
theo.ruys@axini.com
Axini B.V.
Amsterdam, The Netherlands

Abstract

In Model-Based Testing (MBT), long traces to errors complicate bug localisation. Shrinking of these long traces, aimed at finding smaller counterexamples from existing ones, traditionally operates at the trace-level. However, leveraging the detailed behaviour captured by MBT models suggests potential for more effective shrinking methods. We propose a model-centric shrinking approach, the *Shortest-PathShrinker*. It centres around the idea that finding shorter traces in MBT is similar to finding shorter paths in graphs. Our experimental results show that the algorithm yields smaller or comparable traces to existing methods, in a faster fashion depending on the type of bug. Future work is discussed.

CCS Concepts

• Software and its engineering → Software testing and debugging.

Keywords

model-based testing, automated debugging, failure analysis

ACM Reference Format:

Susan van den Broek, Ana Oprescu, and Theo C. Ruys. 2024. Bug Localisation in Model-Based Testing using Model-Centric Shrinking. In *Proceedings of the 2nd ACM International Workshop on Future Debugging Techniques (DEBT '24)*, September 19, 2024, Vienna, Austria. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3678720.3685315>

can be found that would be harder to catch using unit testing [17]. The downside to this, is that the traces can get very long, and practitioners mention that failing test traces of 1000 steps are not uncommon. While these traces are automatically generated, the analysis of their failure is not automated. Bug localisation is done by the human developer, and not by the MBT suite.

Bug localisation is one of the most tedious and costly activities of the debugging process [31], and one of the challenges of debugging is cognitive overload [33]. It would thus be beneficial to developers and testers if a shorter trace to the same error could be produced, as this makes the analysis of the traces and therefore bug localisation easier. This procedure is called *shrinking*, and it enables to find smaller counterexamples based on an already found counterexample [12].

Several techniques to achieve shrinking exist, and most of these methods utilise an iterative approach to step-wise shrink the failing test trace. At each iteration, the shrinking algorithm uses its heuristic or bug localisation method to decide what part of the test trace to remove. Then, the shrunk trace is put through the MBT engine again in order to see if it produces the sought-for error. Because these methods depend entirely on the original failing trace, they are limited in the alternative shorter traces that they can produce. Alternatively, because the *sur* is invoked at each iteration, these shrinking methods can be slow when they require many iterations to get to the smallest trace [14]. Industry practitioners would be helped by a shrinking method that not only produces small traces,

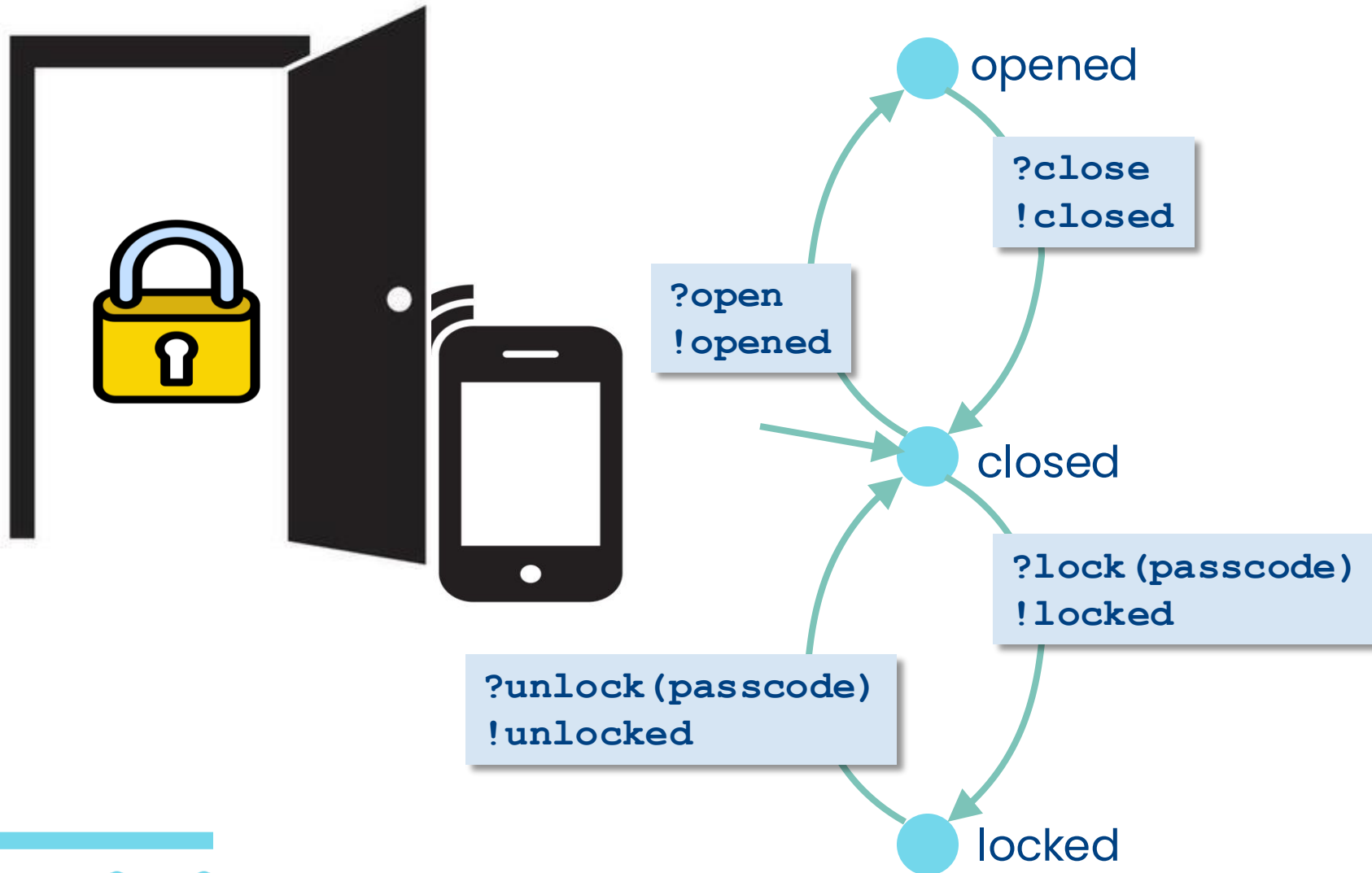
axini

SmartDoor

AMP Bug Hunting Exercise

Laboratory: SmartDoor

Happy Flow behavior of the SmartDoor: all 'invalid' and 'incorrect' labels are **not included**.



After three incorrect passcodes, the SmartDoor will 'shut off'.

Laboratory: SmartDoor

- **SmartDoor**
 - develop a **model** on basis of a specification
 - using a (correct) SUT to develop and **test** the model
 - **testing** several (incorrect?) SUTs
 - **find all the bugs** in the incorrect implementations
- **Material**
 - SmartDoor **specification**: formal specification
 - **Laboratory text**: roadmap to approach the exercise

Carefully study the **specification** before following the **Laboratory steps**.

Remember: goal is to make a **model of the SUT**.

- alternative, **high-level abstraction** of the SUT
- **direction of messages** (stimuli, responses) is from the point of view of the SUT.

Laboratory: SmartDoor – HOWTO

2. SmartDoor

Now that you are more familiar with AMP, we can continue to the main event of this laboratory. You will do a small MBT project in which you will model and test various implementations of controlled door called SmartDoor. This project should give you a good feel for how MBT practice.

In AMP, you can select the test sets from the top. This page contains multiple Test Sets for the SmartDoor project. Choose the Axini test set.

2.1 Modeling

In this exercise, you will write a model in AML from scratch.

Informally, the door has the following features: it can be opened, closed and locked. When it is locked, a passcode is set. A separate *SmartDoor Interface Requirements Specification* document is provided that contains more details.

In the following sections, we will guide you through modeling the behavior of SmartDoor. In each step, we recommended checking the visualization to see if your model ended up the way intended. Moreover, the SmartDoor implementation by Axini has already been tested with can consider this implementation to be correct and use it to validate your model.

Be sure to model **all requirements** of the "official" **specification**.

Follow the **step-wise instructions** in the exercise **description**.

SMARTDOOR Interface Requirements Specification

SmartDoor Solutions

IRS Version 2.0
September 2020

*Warning: in certain places, the requirements may **not be precise** enough... as with specifications in practice.*

AML Quick Reference Card

For the 'SmartDoor' exercise, only the following statements are needed:

- **send & receive**
- **choice**
- **repeat**
- **state & goto**

+ variables, constraints, and updates

```
external channel_name [, type: :duplex]
internal channel_name
...
process(process_name) {
  timeout <decimal_value>
  channel(channel_name) {
    stimulus label_name ,
      param_name => <type> ,
      ...
    response label_name, ...
  }
  var var_name, <type> [, <initial_value>]
  ...
  # behavior of process: AML statements
}

process(process_name) {
  ...
}
```

channel_name has to be declared globally

AML constructs are separated by newlines or ;

include 'model_part.aml' can be used anywhere

<type>

:integer
:decimal
:boolean
:string
:date
:time

send
receive

expedit
urge
aft
befo
constrai
upda
no

<statements> ::
| { <state

[<type>] list
{ <type> => <type> } hash
{ field_name => <type> , struct

choice {
o <statements>
o <statements>
...
}

state state_name
goto state_name

repeat {
o <statements>
o <statements>
...
}

stop_repetition
next_repetition

if <bool_expr> ,
then { ... } [,
else { ... }]

deterministic loop

while(<bool_expr>) {
... # body
}

optionally { <statements> }

internal *constraint*

constraint <bool_expr>

internal *action*

update <assignment_expr>

Ruby code

```
def method(params)
  ...
end
```

```
if <ruby_expr>
  # AML fragments
else
  # AML fragments
end
```

```
"...#{<ruby_expr>}..."
```

- **Names** of labels, states, variables are 'strings' in quotes.
- Curly brackets { ... } are used to group statements.
- Statements are separated by **newlines** (or ';').

<type>) { |p, ...|

expressions: constraints or updates

fixed font keywords
*_name string, e.g. "Notify"
<*_expr> string, e.g. "x > 5"
<foo> grammar placeholder
... repetition / placeholder
[...] optionally

Test Sets: working together on same account

- **Projects** for each UvA group:
 - Coffee Machine
 - SmartDoor
- A project can have **multiple Test sets**.
- For each **Test set** you have to modify:
 - **root model part**: the "main" AML file
 - **configuration** (number of test cases, steps, strategy, etc.)
- **Git** can also be used (versions and branches), but is not needed.

***Test Sets** can be seen as **different views** on same the project. For example: Master model, development branches, Bad Weather, Replaying bugs, etc.*

Axini Piet

smartdoor-piet.aml

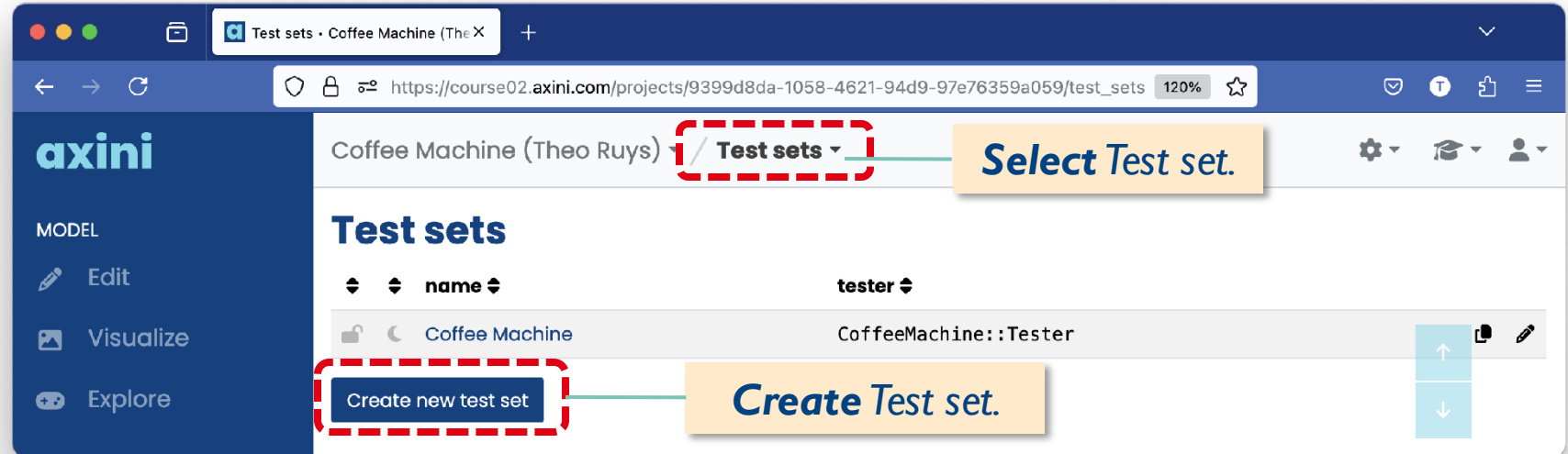
Axini Hein

smartdoor-hein.aml

Test Sets: working together on same account

1

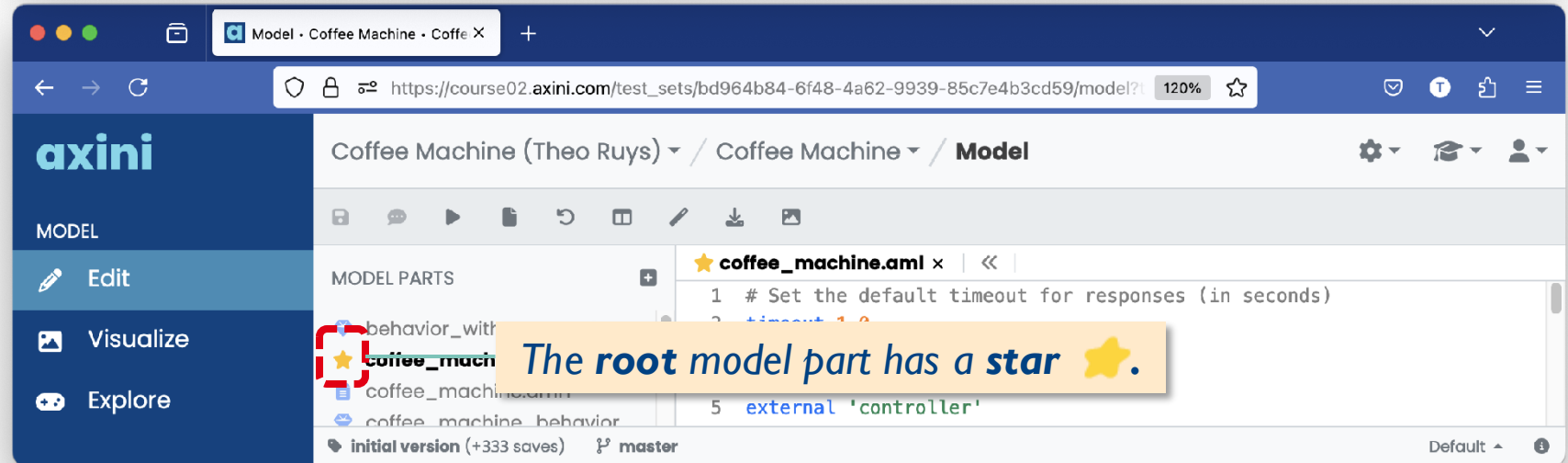
Create or
Select **Test set**.



2

Set the **root model part** for the Test set.

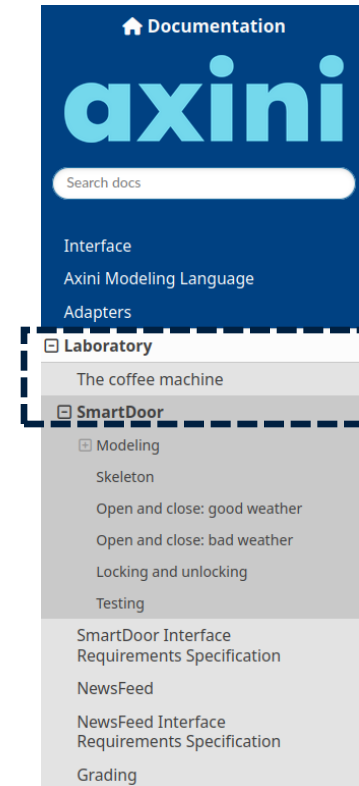
*Right-click on a file to make it the **root model part** of the current Test Set.*



+ (optionally) change the **configuration**

Don't know how to start?

1. Go to course02.axini.com; log in
2. Open the documentation; go to the SmartDoor laboratory
3. Inside the main app, open the SmartDoor project, 'Axini' test set
4. Click 'edit' on the right
5. Start implementing the 'Skeleton' of the model



interactions with the SUT are possible. As you will see, some cases cannot be tested with the given adapter.

The door will always begin *closed* and *unlocked*. The adapter ensures the SUT is in this state when a test case is started.

Skeleton

First, we will write a skeleton for the SmartDoor model, making use of what you have learned during the Coffee Machine exercise.

- Define a **process** for the door and an **external channel** named `door`.
- Define the labels needed for the model. The label and channel names cannot be chosen freely as they must be known by the adapter used by the SUT. The configured adapter implements the labels as written in the specification on the channel named `door`. Start with the stimuli `open` and `close` and the responses `opened` and `closed`.
- Optionally, you could also define the other stimuli and responses at this stage. This might save some time later on.
- Set a global **timeout** for responses.

Open and close: good weather

When modeling a new system, we generally start small and extend the model over time. The SmartDoor model is well suited for this approach.

- Let us start with modeling some of the **good weather** behavior. This is the intended usage of the system. Add an AML fragment that describes the opening and closing behavior of the door. You can use the same modeling concepts used in the Coffee Machine model.

axini

SmartDoor: Some AML Tips

Some common mistakes

- The SmartDoor SUTs start in a **closed** + **unlocked** state
- Only use the label names as mentioned in the **text**. So:

```
stimuli    'open', 'close'  
stimulus   'lock', { 'passcode' => :integer }  
responses  'opened', 'closed', 'invalid_command'
```

- Constraints and updates are **strings**: '...' or "..."

```
receive 'lock', constraint: 'passcode >= 0 && passcode <= 9999' ,  
          update: "entered_passcode = passcode"
```

- Only a **single** constraint and update per send/receive.

- not: `send 'foo', update: 'x = 1', update: 'y = 2'`

- but: `send 'foo', update: 'x = 1; y = 2'`

- Deterministic choice

```
_if 'x > 0' ,  
_then { send 'positive' } ,  
_else { send 'negative' }
```

Both the `_if` and `_then` and the `_then` and `_else` should be separated by a **comma**.

For the SmartDoor, there is only one situation where `_if _then _else` is appropriate.

- The **passcode** is **dynamic**

Think of it as the lockers at the gym

Using Ruby as preprocessor

```
config = {  
  :include_lemonade => true  
}
```

Ruby fragment

```
process('main') {  
  channel('controller') {  
    stimuli 'button_tea', 'button_coffee', 'button_lemonade'  
    responses 'tea', 'coffee', 'lemonade'  
  }  
}
```

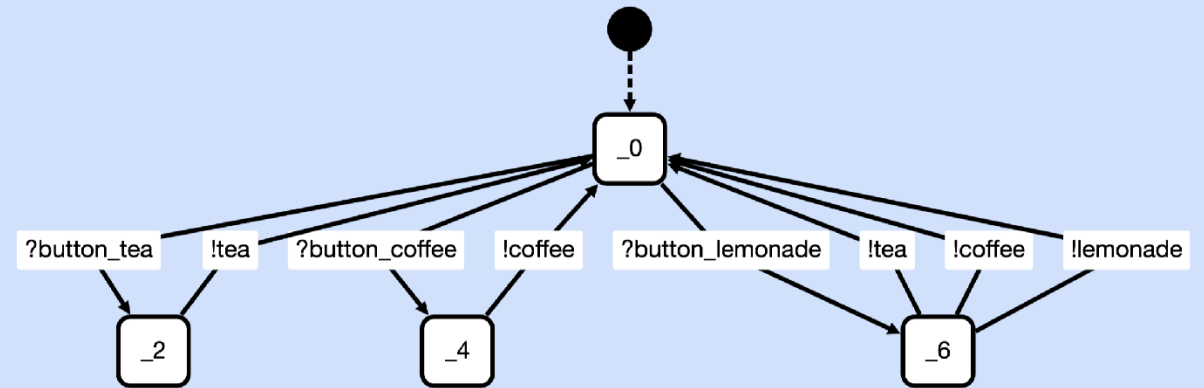
```
repeat {  
  o { receive 'button_tea' ; send 'tea' }  
  o { receive 'button_coffee' ; send 'coffee' }
```

```
  if config[:include_lemonade]  
    {  
      receive 'button_lemonade'  
      choice {  
        o { send 'tea' }  
        o { send 'coffee' }  
        o { send 'lemonade' }  
      }  
    }  
}
```

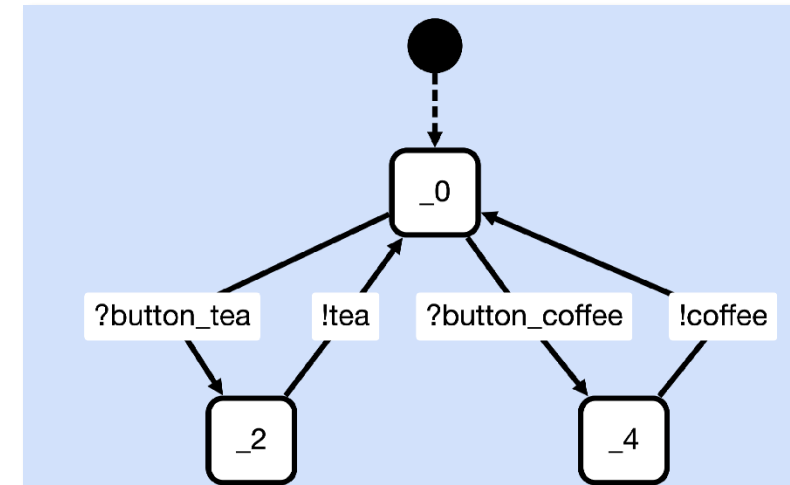
```
end  
}
```

*Similar to the CPP preprocessor
which is used in C/C++:
#define #if #ifdef, etc.*

config[:include_lemonade] == true



config[:include_lemonade] == false



SmartDoor – some modeling tips (1/2)

To **include/exclude** model **fragments**:

Add the following **Ruby hash** to the preamble of your model:

```
config = {  
  :besto_1 => false,  
  :besto_2 => true,  
  :logica_1 => false,  
}
```

... or the equivalent:

```
config = {  
  besto_1: false,  
  besto_2: true,  
  logica_1: false,  
}
```

Within the model itself:

```
if config[:besto_1]  
  ... # AML fragments to include  
end  
  
...  
  
if config[:logica_1]  
  ... # AML fragments: Logica bug  
else  
  ... # AML fragments: correct behavior  
end
```

This ensures that we have a **single AML model** which contains all expected behavior and all bugs found.

SmartDoor – some modeling tips (2/2)

- To observe **'quiescence'** in the model: *needed for SECLOC-07*

Add **'virtual'** response 'quiescence' to the process.

```
response 'quiescence', on: 'external'
```

This response will **never arrive**, so AMP will wait for 2.0 seconds.

```
optionally { send 'quiescence', timeout: 2.0 }
```



```
choice {  
  o { send 'quiescence',  
        timeout: 2.0 }  
  o {}  
}
```

- Use **notes** to emphasize specific labels in a test case (trace):

```
send 'opened', note: 'opened'
```

```
send 'locked', note: ['locked', '$entered_passcode #red']
```

*inserts the contents of state **variable** 'entered_passcode' into the **note***

You can also use **label parameters** of the label here.

Laboratory: Report

Report of the bugs should **not be longer** than a single **A4** paper.

- Write a small **report** on the **bugs** found in the various SUTs.
 - Do **not** go **overboard** here! We are only **interested** in the **bugs found** and a concise and precise **description** of these bugs.

Section 2.6
of the
Laboratory
text.

For you, it is enough if you can answer the following questions.

1. In what state(s)/situation(s) does the bug occur?
2. What is the expected behavior?
3. What behavior is observed instead?
4. What is your hypothesis of how the bug works?

Formulate the bug in **one** (or two) **sentences** (with a reference to the violated requirement).

For example:

Micro\$oft SUT: when locked, after an ?open stimulus, the SUT responds with a !unlocked response, whereas it should have responded with an !invalid_command (see: BEHAVR-01 and BEHAVR-05).

What if you need help?

- Click ☰ → Documentation
- Really stuck?
Email students@axini.com
- Mention the University of Amsterdam

The image shows two overlapping screenshots of the Axini Modeling Platform interface. The top screenshot displays the 'Test runs' page for 'SmartDoor (TU Delft 2026 - Group 1)'. It features a left sidebar with 'MODEL' (Edit, Visualize, Explore) and 'TEST' (Test runs, Configure) sections. The main content area shows 'Test runs' with buttons for 'Start test run', 'Delete selected', and 'Scheduled test runs'. A right sidebar contains 'Campaign', 'Reports', 'Activity', and user settings. The bottom screenshot shows the 'Documentation' page, which includes a search bar, a list of topics (Interface, Axini Modeling Language, Adapters, Laboratory, Frequently asked questions), and a 'Next' button. The documentation content welcomes users to the Axini Modeling Platform (AMP) and lists key features.

axini

SmartDoor (TU Delft 2026 - Group 1) / Axini / Test runs

Start test run Delete selected Scheduled test runs

Test runs

There are no test runs associated with this test set.
Start one using the button above!

MODEL

- Edit
- Visualize
- Explore

TEST

- Test runs
- Configure

Documentation

axini

Search docs

Interface

- Axini Modeling Language
- Adapters
- Laboratory
- Frequently asked questions

axini

Home / Axini Modeling Platform View page source

Axini Modeling Platform

Welcome to the documentation portal of the **Axini Modeling Platform (AMP)**, the platform for model based testing and working.

- Interface describes the various features of the Axini Modeling Platform web interface.
- Axini Modeling Language contains all there is to know about modeling with the Axini Modeling Language.
- Adapters describes how to write (plugin) adapters.
- Laboratory contains all documents handed out during courses and laboratories
- Index lists all terms and definitions
- Frequently asked questions The FAQs about MBT and AMP

Next

© Copyright 2026, Axini B.V..
Built with Sphinx using a theme provided by Read the Docs.